

DAA - Design and Analysis of Algorithms

Session 2 Lab Experiments

Name: J O Joanna Divine

Reg No: 192572003

Question 1: Unique Elements from a List (Space Complexity)

Problem: Take a list of n numbers and create a new list containing only the unique elements from the original list. State the space complexity of the algorithm.

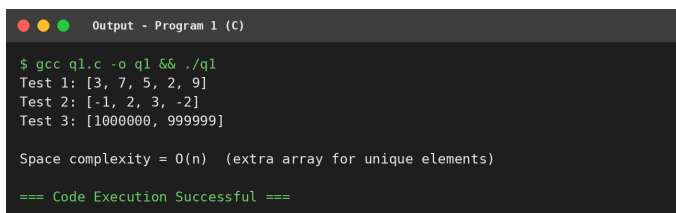
Program (C):

```
#include <stdio.h>

void printUnique(int arr[], int n) {
    int result[100];
    int count = 0, i, j;
    for (i = 0; i < n; i++) {
        int duplicate = 0;
        for (j = 0; j < count; j++) {
            if (result[j] == arr[i]) {
                duplicate = 1;
                break;
            }
        }
        if (duplicate == 0) {
            result[count] = arr[i];
            count++;
        }
    }
    printf("[");
    for (i = 0; i < count; i++) {
        printf("%d", result[i]);
        if (i != count - 1)
            printf(", ");
    }
    printf("]\n");
}

int main() {
    int a[] = {3, 7, 3, 5, 2, 5, 9, 2};
    int b[] = {-1, 2, -1, 3, 2, -2};
    int c[] = {1000000, 999999, 1000000};
    printf("Test 1: ");
    printUnique(a, 8);
    printf("Test 2: ");
    printUnique(b, 6);
    printf("Test 3: ");
    printUnique(c, 3);
    printf("\nSpace complexity = O(n) (extra array for unique elements)\n");
    return 0;
}
```

Output:



```
Output - Program 1 (c)

$ gcc q1.c -o q1 && ./q1
Test 1: [3, 7, 5, 2, 9]
Test 2: [-1, 2, 3, -2]
Test 3: [1000000, 999999]

Space complexity = O(n) (extra array for unique elements)

=== Code Execution Successful ===
```

Complexity: Time: $O(n^2)$ (each element is checked against the result list). Space complexity: $O(n)$ - the new list can hold up to n distinct elements.

Question 2: Bubble Sort with Big-O Analysis

Problem: Sort an array of integers using the bubble sort technique and analyse its time complexity using Big-O notation.

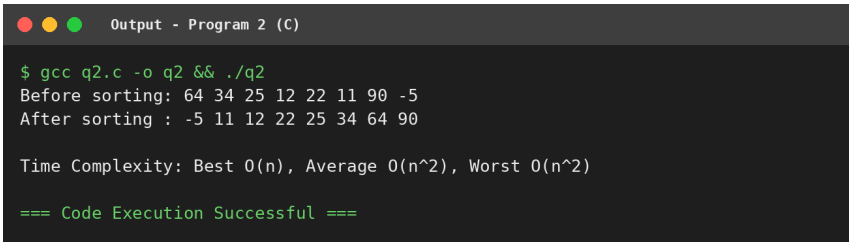
Program (C):

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    int i, j, temp, swapped;
    for (i = 0; i < n - 1; i++) {
        swapped = 0;
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }
        if (swapped == 0)
            break;
    }
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90, -5};
    int n = 8, i;
    printf("Before sorting: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    bubbleSort(arr, n);
    printf("\nAfter sorting : ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n\nTime Complexity: Best O(n), Average O(n^2), Worst O(n^2)\n");
    return 0;
}
```

Output:



```
Output - Program 2 (C)

$ gcc q2.c -o q2 && ./q2
Before sorting: 64 34 25 12 22 11 90 -5
After sorting : -5 11 12 22 25 34 64 90

Time Complexity: Best O(n), Average O(n^2), Worst O(n^2)

=== Code Execution Successful ===
```

Complexity: Best case $O(n)$ (already sorted, thanks to the swapped flag), Average $O(n^2)$, Worst $O(n^2)$. Space: $O(1)$.

Question 3: Binary Search for Element x

Problem: Check whether a given number x exists in a sorted array arr using binary search and analyse its time complexity using Big-O notation.

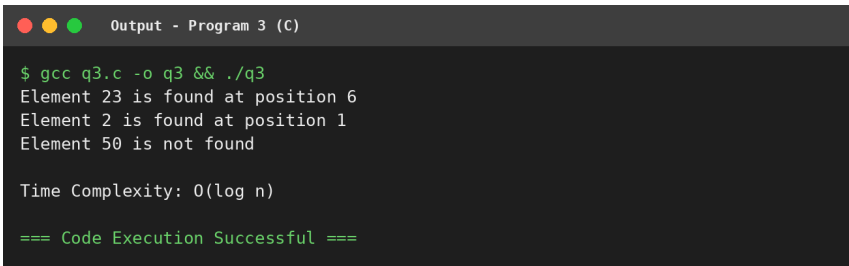
Program (C):

```
#include <stdio.h>

int binarySearch(int arr[], int n, int x) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (arr[mid] == x)
            return mid;
        else if (arr[mid] < x)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main() {
    int arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    int n = 10, keys[3] = {23, 2, 50}, i, pos;
    for (i = 0; i < 3; i++) {
        pos = binarySearch(arr, n, keys[i]);
        if (pos != -1)
            printf("Element %d is found at position %d\n", keys[i], pos + 1);
        else
            printf("Element %d is not found\n", keys[i]);
    }
    printf("\nTime Complexity: O(log n)\n");
    return 0;
}
```

Output:



```
Output - Program 3 (c)

$ gcc q3.c -o q3 && ./q3
Element 23 is found at position 6
Element 2 is found at position 1
Element 50 is not found

Time Complexity: O(log n)

=== Code Execution Successful ===
```

Complexity: Best case $O(1)$, Average and Worst case $O(\log n)$, because the search range is halved every step. Space: $O(1)$.

Question 4: Sort an Array in $O(n \log n)$ with the Smallest Space Complexity

Problem: Given an array of integers `nums`, sort the array in ascending order without using any built-in functions, in $O(n \log n)$ time and with the smallest possible space complexity. Heap sort is used because it sorts in place.

Program (C):

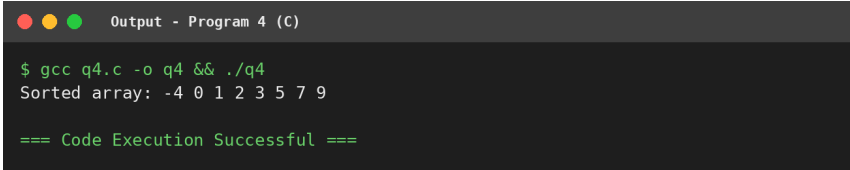
```
#include <stdio.h>

/* heap sort -  $O(n \log n)$  time and  $O(1)$  extra space */
void heapify(int nums[], int n, int i) {
    int largest, left, right, temp;
    while (1) {
        largest = i;
        left = 2 * i + 1;
        right = 2 * i + 2;
        if (left < n && nums[left] > nums[largest])
            largest = left;
        if (right < n && nums[right] > nums[largest])
            largest = right;
        if (largest == i)
            break;
        temp = nums[i];
        nums[i] = nums[largest];
        nums[largest] = temp;
        i = largest;
    }
}

void sortArray(int nums[], int n) {
    int i, temp;
    for (i = n / 2 - 1; i >= 0; i--)
        heapify(nums, n, i);
    for (i = n - 1; i > 0; i--) {
        temp = nums[0];
        nums[0] = nums[i];
        nums[i] = temp;
        heapify(nums, i, 0);
    }
}

int main() {
    int nums[] = {5, 2, 3, 1, 9, -4, 0, 7};
    int n = 8, i;
    sortArray(nums, n);
    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", nums[i]);
    printf("\n");
    return 0;
}
```

Output:



```
Output - Program 4 (C)

$ gcc q4.c -o q4 && ./q4
Sorted array: -4 0 1 2 3 5 7 9

=== Code Execution Successful ===
```

Complexity: Time: $O(n \log n)$ in best, average and worst cases. Space: $O(1)$ - in-place, iterative heapify.